

Լաբորատոր աշխատանք 3

Նպատակ (Աշխատանքի նպատակ): Աշխատանքի նպատակն է յուրացնել և գործնականում կիրառել դինամիկ զանգվածների, կառուցվածքների (struct), ցուցիչային աշխատանքի, ինչպես նաև հիշողության կառավարման մեխանիզմները (new[], delete[] և nullptr) C++ ծրագրավորման լեզվում:

Խնդրի պահանջ: Ստեղծել C++ ծրագիր, որը կկառուցի ExamResult անունով կառուցվածք՝ ուսանողի անունը, առարկան, գրավոր և բանավոր միավորները պահելու համար. Դինամիկ զանգվածում անհրաժեշտ է պահպանել 12 արդյունք, ֆունկցիայի միջոցով հաշվարկել վերջնական միավորը 60% և 40% կշիռներով, գտնել յուրաքանչյուր առարկայի լավագույն արդյունքը և արտածել չանցած (կտրված) ուսանողներին. Ծրագրում պարտադիր պետք է կիրառվեն դինամիկ զանգված, ցուցիչային աշխատանք, delete[] և nullptr.

Տեսական մաս: Դինամիկ հիշողության բաշխումը C++-ում թույլ է տալիս ծրագրի կատարման ընթացքում (run-time) հիշողություն հատկացնել զանգվածներին կամ օբյեկտներին՝ օգտագործելով new[] օպերատորը: Կառուցվածքները (struct) հնարավորություն են տալիս միավորել տարբեր տիպի տվյալներ մեկ ընդհանուր տիպի մեջ: Ցուցիչների հետ աշխատելիս կարևոր է հիշողության արտահոսքից (memory leak) խուսափելու համար ազատել հատկացված տարածքը delete[] հրամանով, այնուհետև ցուցիչին վերագրել nullptr արժեքը՝ անվտանգության նկատառումներով: Վերջնական միավորի հաշվարկը կատարվում է բանաձևի միջոցով, որտեղ գրավոր և բանավոր քննությունների արդյունքները բազմապատկվում են համապատասխանաբար 0.60 և 0.40 գործակիցներով:

```

struct ExamResult {
    std::string name;
    std::string subject;
    double writtenScore; // Գրավոր միավոր
    double oralScore;    // Բանավոր միավոր
};

// Ֆունկցիա՝ վերջնական միավորը հաշվելու համար (60% գրավոր + 40% բանավոր)
double calculateFinalScore(const ExamResult* result) {
    return (result->writtenScore * 0.6) + (result->oralScore * 0.4);
}

// Ֆունկցիա՝ յուրաքանչյուր առարկայի լավագույն արդյունքը գտնելու համար
void printBestPerSubject(const ExamResult* results, int size) {
    std::cout << "\n=== BEST RESULT PER SUBJECT ===\n";

    // Գտնում ենք չկրկնվող առարկաները և դրանց առավելագույն միավորները
    for (int i = 0; i < size; ++i) {
        bool alreadyProcessed = false;
        for (int j = 0; j < i; ++j) {
            if ((results + i)->subject == (results + j)->subject) {
                alreadyProcessed = true;
                break;
            }
        }

        if (!alreadyProcessed) {
            std::string currentSubject = (results + i)->subject;
            const ExamResult* bestStudent = nullptr;
            double maxScore = -1.0;

            for (int k = 0; k < size; ++k) {
                if ((results + k)->subject == currentSubject) {
                    double score = calculateFinalScore(results + k);
                    if (score > maxScore) {
                        maxScore = score;
                        bestStudent = results + k;
                    }
                }
            }
        }
    }
}

```

```

    }
    }
}

if (bestStudent != nullptr) {
    std::cout << "Subject: " << std::left << std::setw(15) << currentSubject
        << " | Best Student: " << std::setw(10) << bestStudent->name
        << " | Score: " << maxScore << "\n";
    }
}
}

// Ֆունկցիա՝ չանցած (վերջնական միավորը < 40) ուսանողներին արտածելու
համար
void printFailedStudents(const ExamResult* results, int size, double passThreshold =
40.0) {
    std::cout << "\n=== FAILED STUDENTS (Final Score < " << passThreshold << ")
===\n";
    bool foundFailed = false;

    for (int i = 0; i < size; ++i) {
        const ExamResult* current = results + i; // Ցուցիչային հասցեագրում
        double finalScore = calculateFinalScore(current);

        if (finalScore < passThreshold) {
            std::cout << "Student: " << std::left << std::setw(10) << current->name
                << " | Subject: " << std::setw(15) << current->subject
                << " | Score: " << finalScore << "\n";
            foundFailed = true;
        }
    }

    if (!foundFailed) {
        std::cout << "No failed students found.\n";
    }
}

int main() {
    const int count = 12;

```

```

// 1. Դիսամիկ հիշողության հատկացում 12 արդյունքի համար
ExamResult* results = new ExamResult[count]{
    {"Aram", "Mathematics", 85, 90},
    {"Ani", "Physics", 30, 40},
    {"Davit", "Mathematics", 95, 80},
    {"Meri", "Programming", 70, 75},
    {"Gor", "Physics", 25, 30},
    {"Lilit", "Programming", 90, 95},
    {"Karen", "Mathematics", 40, 35},
    {"Sona", "Physics", 88, 92},
    {"Armen", "Programming", 35, 45},
    {"Elen", "Mathematics", 60, 65},
    {"Vahe", "Physics", 50, 45},
    {"Hasmik", "Programming", 100, 98}
};

// 2. Մշակված գրառումների արտածում
std::cout << "=== ALL PROCESSED RECORDS ===\n";
for (int i = 0; i < count; ++i) {
    const ExamResult* ptr = results + i; // Ցուցիչով աշխատանք
    std::cout << std::left << std::setw(2) << i + 1 << ". "
        << "Name: " << std::setw(10) << ptr->name
        << " | Subject: " << std::setw(15) << ptr->subject
        << " | Written: " << std::setw(3) << ptr->writtenScore
        << " | Oral: " << std::setw(3) << ptr->oralScore
        << " | Final: " << calculateFinalScore(ptr) << "\n";
}

// 3. Յուրաքանչյուր առարկայի լավագույն արդյունքի որոնում և արտածում
printBestPerSubject(results, count);

// 4. Չանցած ուսանողների արտածում
printFailedStudents(results, count);

// 5. Հիշողության ազատում delete[]-ով և ցուցիչի վերագրում nullptr-ի
delete[] results;
results = nullptr;
return 0;
}

```